

Docker

Concepts to be discussed:

Types of applications:

- > Purpose of learning application types and architecture.
- > Standalone application
- > web-based application
- > Client server application

Types of application architecture:

- > Monolithic
- > 3-tier architecture
- > Microservices

Introduction to containers:

- > Physical machines vs Virtual machines vs Containers
- > What are containers?

Introduction to Docker:

- > What is Docker?
- > History and features
- > Alternative tools
- > Docker architecture
- > Basic commands

Dockerfiles:

- > Dockerfile examples

Multi-stage Dockerfiles:

- > Why do we need multi-stage Dockerfile?
- > Multistage dockerfile implementation

Docker Volumes

- > Why do we need Docker volumes?
- > Types of docker volumes
- > Basic commands in docker volumes

Docker Networks

- > Why do we need Docker network?
- > Types of docker network
- > Basic commands in docker network

Docker Compose

- > Why do we need docker compose?
- > Introduction to YAML file
- > Docker Compose YAML file example

Integrating Jenkins with Docker

Introduction to ECS and ECR

Purpose of learning application types and architecture:

Before we talk about Docker or containers, we must first understand what kind of application we are trying to deploy and how they are structured.

Types of application:

1) Standalone application:

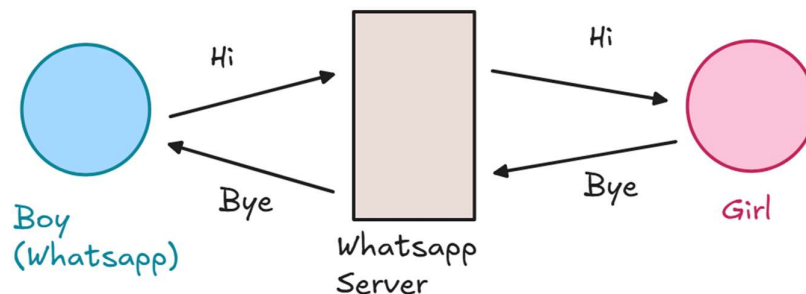
- > A standalone application runs independently on a single computer. It does not require an internet connection or external server to function once installed.
- > It is installed locally on a user's computer.
- > It uses system resources such as CPU, RAM and some of the local files.
- > Updates needs to be done manually on each machine.
- > Examples: paint, File explorer, calculator, Offline Games etc.....

2) Web-based application:

- > A web-based application runs on a web server and is accessed through web browser using a URL. It does not need any installation on the user's device.
- > It is a type of application which requires stable internet connection.
- > It is accessible over HTTP or HTTPS.
- > Easy to update since all the changes happen on the server.
- > Example: AWS, Jenkins, SonarQube.....

3) Client-server application:

- > A client-server application is one where the client sends the request, and the server processes it and sends back a response. The key concept is that the communication happens over a network.
- > Clients communicate with servers using protocols like HTTP or TCP/IP.
- > It can multi-tier.
- > It is scalable.



Types of application architecture:

1. Monolithic Architecture:

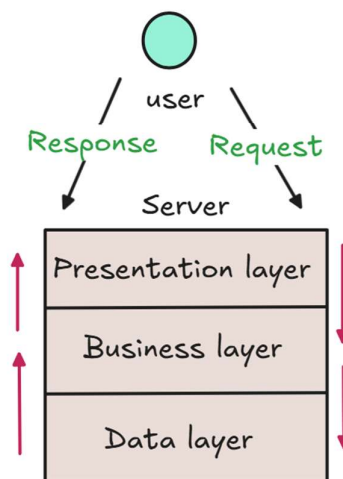
- > A monolithic application is built as one single, large unit. All parts - The user interface, Business logic and the database are combined and deployed together as one application.

How it works?

- > A user sends a request (Example, logging in or buying an item).
- > The entire application receives the request.
- > Inside the same codebase, the request is processed through the user interface logic, business logic and database operations.
- > The final response is sent back to the user.

Disadvantages:

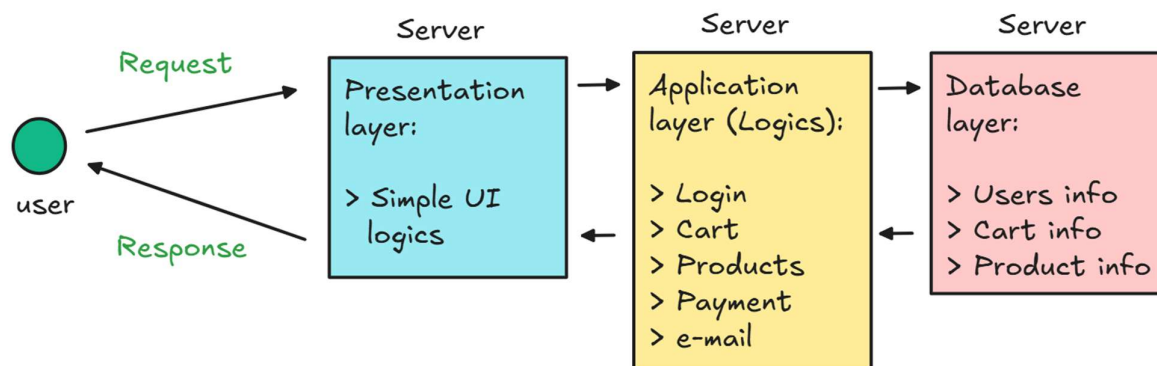
- > If one small part fails, the whole system can crash.
- > To make any change, you must rebuild and redeploy the entire application.
- > Hard to scale specific parts.
- > Code becomes messy and tightly coupled (dependent on each other) as it grows.



2. 3-tier architecture

Instead of keeping everything mixed, 3-tier architecture divides the app logic into 3 layers:

- > Presentation layer: Handles user interface (User Interface using HTML, CSS.....)
- > Application layer: Handles business logic (Like, Java, Node.js, python.....)
- > Data layer: handles database operations (Like MySQL, MongoDB, Oracle DB.....)



How it improved over monolithic:

- > Better structure: Clear separation of layers
- > Easier to maintain or replace one layer
- > Still single deployable unit, but more organised.

Disadvantages:

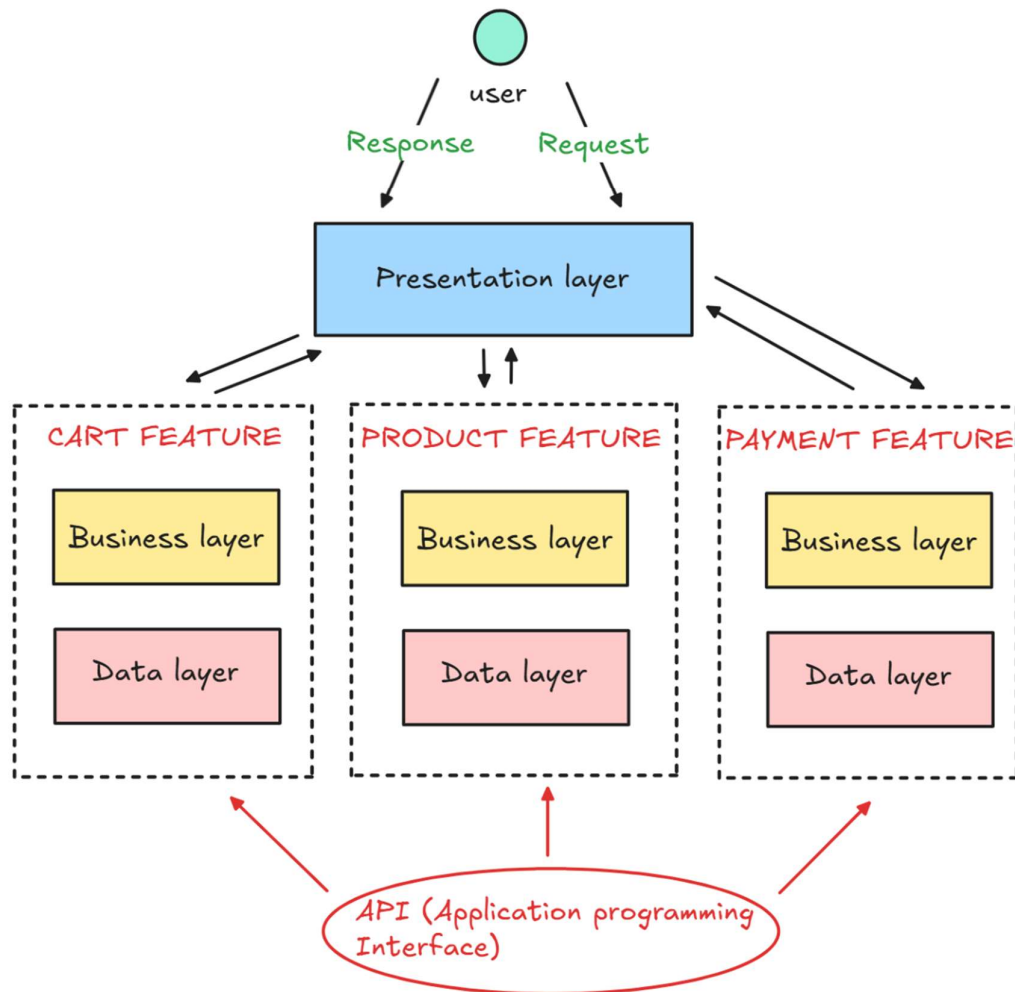
- > Slightly slower due to multi-layers
- > Can't scale individual features.
- > Still the features are tightly coupled (Like, Cart, product, payment

3. Microservices architecture:

- > A microservices architecture is when an application is divided into many small independent services and each service performs a specific function.

Each microservice:

- > Has its own codebase.
- > Can be developed, tested and deployed separately.
- > Often has its own database.
- > Communication happens through APIs



Advantages:

- > Each service can be developed and deployed independently.
- > If one service fails, other services will still run.
- > You can scale only what's needed (Ex: If orders increase, we can add more servers only to handle orders).
- > Teams can use different programming languages or databases for different services.

Disadvantages:

- > Managing and monitoring many services is complex.
- > Needs solid DevOps tools.
- > APIs must be well-designed.
- > Each service may need a different environment.

Introduction to containers

1. Physical machines:

- > A physical server is a single dedicated machine, that runs one OS directly on it's hardware.
- > Applications are installed directly on this physical machines.

Problems:

- > Low resource utilization: The CPU, RAM or storage may not be fully utilized.
- > High cost: Each new application may need a physical server and cost of buying a physical server is more.
- > Scalability: You can't easily add or remove resources.
- > Environment conflicts: Two apps needing different dependencies can't co-exist.

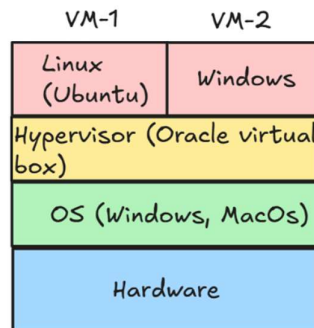
2. Virtual machines (Virtualization):

- > Virtualization is the technology that allows multiple operating systems to run on a single physical server using a software layer called as Hypervisor. each of it's OS will run inside it's own VMs.

Advantages:

- > Resources are properly utilized.
- > You can run multiple OS at once.
- > It is easier to isolate and manage the applications.

Configuration:
30 GB -----> storage
4 GB ----> RAM



Configuration:
10 GB -----> storage
2 GB ----> RAM

Configuration:
512 GB -----> storage
8 GB ----> RAM

Disadvantages:

- > Each VM needs it's own OS, which consumes memory and storage.
- > Boot time is slow because every VM behaves like a full computer.
- > Number of VMs are dependent on base configuration.
- > The file we are getting from VMs are more in size.

3. Containers:

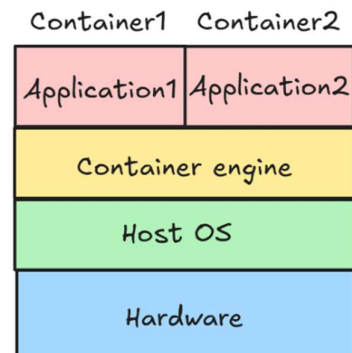
- > Containers are light-weight, standalone environments that package an application with all it's required dependencies, libraries and configurations.
- > But instead of running a full OS, they share the host OS kernel.

How containers works?

- > Containers isolate application from each other, but they all share the same OS.
- > Containers start instantly because there is no need to boot a full OS.

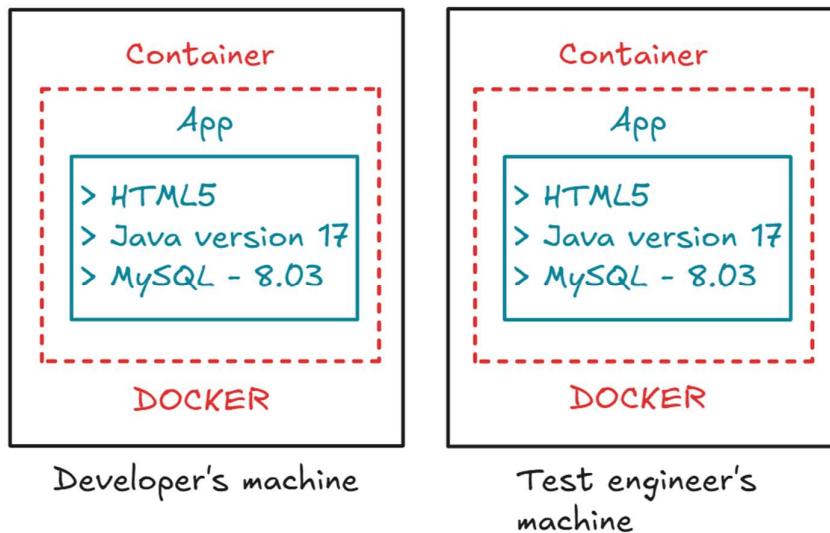
Advantages:

- > Very lightweight and fast.
- > No need for a full OS inside each container.
- > Perfect for microservices architecture, where each service runs inside it's own container.



Introduction to Docker:

> Docker is a containerization platform that allows you to package an application and all its dependencies into a single unit called as container, so it runs consistently across any environment.



History of Docker:

2010: Solomon Hykes invented dotCloud where he was able to create containers

2013: Docker was invented as a free and open source platform where we can create containers for storing the applications.

2014: Docker Inc launched "DockerHub" for storing the images for the containers. Along with this, Docker swarm and Docker Compose got introduced.

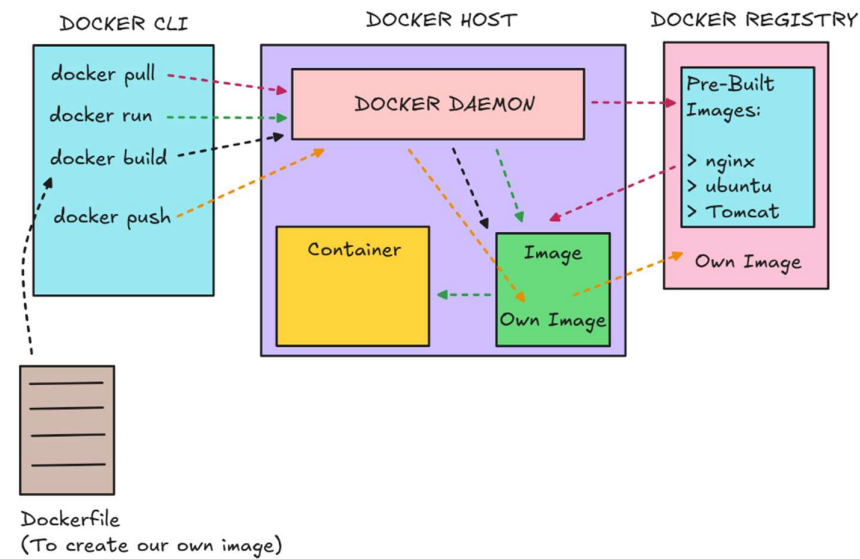
Features:

- > Lightweight: Shares the Host OS, and uses fewer resources than VMs.
- > Portable: Runs consistently across any environment or OS.
- > Isolated: Each container runs independently.
- > Speed: Containers start and stop within seconds.
- > Consistency: Containers show the same behaviour across different environment (Development, testing and production).
- > Free and open source
- > It has DockerHub for storing Images.
- > It can be integrated with other tools like Jenkins, Kubernetes etc

Alternative tools:

1. Podman
2. Containerd
3. Swarm
4. rkt (Rocket)

Docker Architecture:



Docker follows a client-server architecture. It has 3 main components:

- 1) Docker CLI: Tool used to interact with Docker (Through commands)
- 2) Docker Host (Daemon): Docker Daemon is a background service that does all the work ----> It builds, Runs and manages the containers.
- 3) Docker registry: Storage place for Docker images (Docker Hub).

Docker client (Docker CLI):

- > It is a command line interface for Docker.
- > You can use it to run commands like:

```
docker pull
docker run
docker build
docker push
```

- > The client communicates with the Docker Daemon.

Docker Daemon (Docker engine):

- > It is a core service that runs in the background;
- > It does all the heavy work like:
 - > Building the images
 - > Managing networks and volumes
- > It listens the commands form the Docker client and executes them.

Docker images:

- > It is a templete used to create containers.
- > It contains:
 - > Application code
 - > Runtime environment
 - > Dependencies
- > It contains the instructions to run the app.
- > Images are built using Dockerfile, which contains step-by-step instructions.
- > If any changes or modification happens to the existing image then it will be considered as the new image.

Docker Containers:

- > Docker containers are the running instance of an image.
- > It is isolated from other containers and runs on a host machine using shared OS.
- > Containers can be :
 - > started
 - > stopped
 - > restart
 - > delete

Docker Registry:

- > Docker registry stores and distributes docker images.
- > The most popular docker registry is DockerHub.
- > You can also create private registries for you organisation..
- > We can get pre-built images like ubuntu, nginx, tomcat
- > If required we can push our own images into the registry.

Basic Commands:

- > **docker pull:** This command is used to pull the images from DockerHub
Example: `docker pull nginx:latest`
- > **docker images:** This command is used to list all the images available locally on your system.
Example: `docker images`
- > **docker run:** This command is used to create and run the container from the image.
Example: `docker run nginx`
Common flags:
 - > `-d` ---> Run in detached mode (background)
 - > `-it` ---> Interactive mode
 - > `--name` ---> Assigns name for a container
 - > `-p` ---> Maps a container port to a host
 - > `-v` ---> Mount the volume
- > **docker ps:** This command is used to list the running containers
Example: `docker ps`
`docker ps -a`
- > **docker build:** This command is used to build a Docker image from a Dockerfile.
Example: `docker build -t my-img:v1.0 .`
- > **docker rmi:** This command is used to remove one or more images from your system.
Example: `docker rmi my-img:v1.0`
`docker rmi a31gcv59`
- > **docker stop:** This command is used to stop the container.
Example: `docker stop nginx`
`docker stop 1d8s81fg`
- > **docker start:** This command is used to start the container.
Example: `docker start nginx`
`docker start 8cfbd9155`
- > **docker rm:** This command is used to remove the container.
Example: `docker rm nginx`
`docker rm 8cfbd9155`
- > **docker exec:** This command is used to execute a command inside a running container.
Example: `docker exec -it ubuntu-container bash`
- > **docker logs:** This command is used to display the logs of a container.
Example: `docker logs ubuntu-container`
- > **docker rm \$(docker ps -aq):** This command is used to remove all the container.
- > **docker rmi \$(docker images -q):** This command is used to remove all the images.

Dockerfile

- > A Dockerfile is a textfile that contains step-by-step instructions on how to build a docker image.
- > Without Dockerfile, you would have to manually:
 - > Install dependencies
 - > Configure the environments
 - > Copy the code
- > A Dockerfile automates all these tasks.

Common instructions in a Dockerfile:

1. FROM: Defines the base image from which your Docker image will be built.

Syntax: FROM <image>:<tag>

Example: FROM ubuntu:22.04

2. WORKDIR: Sets the working directory inside the image. All the following commands run from this location

Syntax: WORKDIR /path

Example: WORKDIR /app

3. COPY: Copies files from your host system into the image.

Syntax: COPY <source> <destination>

Example: COPY ./pom.xml /app

4. ADD: Similar to COPY but it supports URLs and automatically extracts tar archives.

Syntax: ADD <source> <destination>

Example: ADD https://example.com/file.tar.gz /app

5. RUN: Executes the commands at build time (not when the container runs)

Syntax: RUN <command>

Example: RUN apt update && apt install openjdk-17-jdk -y

6. CMD: Specifies the default command that run when the container starts. CMD is overridden if you pass a command in docker run.

Syntax: CMD ["executable","arg1","arg2"]

CMD <commands>

Example: CMD ["java","-jar","app.jar"]

7. ENTRYPOINT: Similar to CMD but cannot be easily overridden. It defines the "main executable" of the container.

Syntax: ENTRYPOINT ["executable","arg1","arg2"]

Example: ENTRYPOINT ["java","-jar","app.jar"]

8. EXPOSE: Documents which port the container listens on. It does not publish the port by itself.

Syntax: EXPOSE <port>

Example: EXPOSE 8080

9. ENV: Sets the environment variables inside the container

Syntax: ENV key=value

Example: ENV app_name=production

10. LABEL: Adds metadata to your image.

Syntax: LABEL key=value

Example: LABEL maintainer="sumukha"

11. USER: It is used to change the user under which the commands run

Syntax: USER <username or UID>

Example: USER appuser

12. SHELL: It is used to specify the shell to be used inside the container.

Syntax: SHELL ["shell","option"]

Example: SHELL ["/bin/bash","-c"]

Example for static-web-hosting:

- 1) Create a folder and then open VScode from that folder.
- 2) To clone the static website from the GitHub, write these commands:

```
> git clone https://github.com/Sumukha47/Aqua-nova.git  
> cd Aqua-nova
```
- 3) Create a file called as "Dockerfile" and then write these commands:

```
FROM nginx:latest → Base image of nginx  
COPY . /usr/share/nginx/html → Copies the static website files into the nginx directory.  
EXPOSE 80 → Defining the default port number for nginx.
```

Spring-petclinic using java as base image:

```
FROM eclipse-temurin:21.0.9_10-jdk  
WORKDIR /demo  
COPY target/*.jar /demo/petclinic.jar  
EXPOSE 8080  
ENTRYPOINT [ "java", "-jar" ]  
CMD ["petclinic.jar"]
```

Spring-petclinic using maven as base image:

- 1) Create a folder and then open VScode from that folder.
- 2) To clone the spring-petclinic repository, write these commands:

```
> git clone https://github.com/spring-projects/spring-petclinic.git  
> cd spring-petclinic
```
- 3) Create a file called as "Dockerfile" and then write these commands:

```
FROM maven:3.8.5-eclipse-temurin-17 → Base image of maven  
WORKDIR /app → Creates a separate directory called as app. all the other commands  
will be executed within this directory.  
COPY pom.xml /app → Copies the pom.xml into the working directory.  
COPY src /app/src → Copies the src into the working directory and within that inside src folder.  
RUN mvn clean package -DskipTests → Creates a .jar file and skips test phase.  
EXPOSE 8080 → Container port 8080 (default port for spring-petclinic)  
CMD mv ./target/*.jar app.jar && java -jar app.jar → Command to rename the .jar file as app.jar  
and execute jar file.
```

Multi-stage dockerfile

- > A multi-stage Dockerfile is a Dockerfile that contains multiple FROM instructions, where each stage is used for different purpose (Build, test, production, deploy), and only the final output is copied into the final image.
- > Each stage starts from fresh because FROM creates a new build layer.

Why do we use Multi-stage dockerfiles in production?

1. Reduced image size:

Development tools (Maven, Node, JDK) make the image size huge.

> In production you only need JRE.

> Final artifact.

Multi-stage dockerfile removes unnecessary tools.

2. Faster deployment:

> Smaller image -----> Faster push/pull from Dockerhub.

Multistage Dockerfile for spring-petclinic project:

```
# Stage 1: Build the JAR using Maven  
FROM maven:3.8.5-eclipse-temurin-17 AS build  
WORKDIR /app  
COPY pom.xml /app  
COPY src /app/src  
RUN mvn clean package -DskipTests
```

```
# Stage 2: Create lightweight runtime image  
FROM eclipse-temurin:17-jdk-alpine  
WORKDIR /spring-petclinic  
COPY --from=build /app/target/*.jar app.jar  
EXPOSE 8080  
CMD ["java", "-jar", "app.jar"]
```

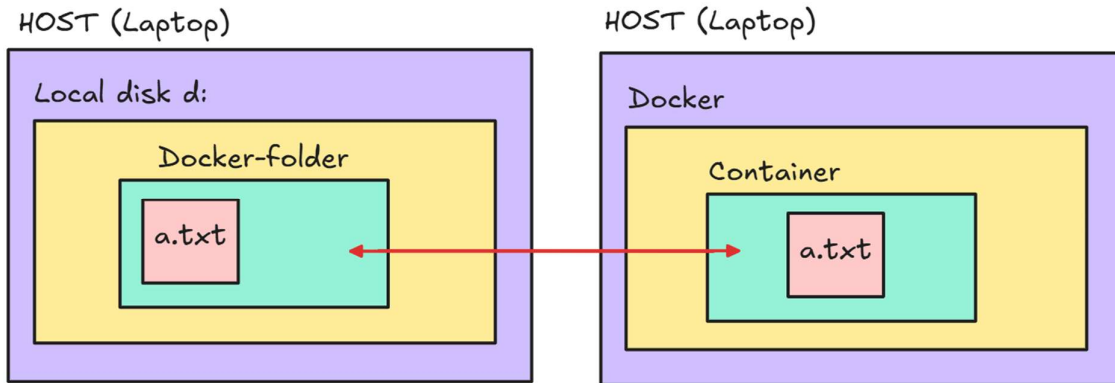
Note: If we have multiple Dockerfiles (like: Dockerfile.dev, Dockerfile.test) then to specifically run a Dockerfile, we use the command:

```
docker build -f Dockerfile.dev -t myapp-dev .  
docker build -f Dockerfile.test -t myapp-test .
```

Docker Volumes

1. Bind mounting:

> Bind mounting is a way of attaching storage to docker container --- but instead of attaching a Docker-managed volume, it directly links a folder from your host machine to the path inside the container.



syntax: `docker run -v /path_on_host:/path_inside_container IMAGE`

Example: `docker run -d -v c:\project\website:/usr/share/nginx/html nginx:latest`

Why Bind mounts are used?

- 1) Local development: When you are developing an application, bind mounts allow you to modify code on your laptop and instantly see the result inside the container.
- 2) Sharing config or sensitive files: Docker should use host machine config files (SSH keys, credentials, SSL certificates...) without copying them to the image.
- 3) Avoid rebuilding images during testing: Everytime you change the code, you shouldn't build the image again. Bind mount solves this.

2. Docker Volumes:

- > Docker Volumes are a mechanism used to store data outside a container so that the data can persist even if the container is deleted, recreated or updated.
- > A Docker Volume is a folder created and managed by Docker where container data is stored permanently.

Docker volume Commands:

1) docker volume create: It is used to create a Docker-managed volume.

Syntax: `docker volume create --name volume_name`

Example: `docker volume create --name myvol`

2) docker volume ls: Lists all the available volumes on the system.

Syntax: `docker volume ls`

3) docker volume inspect: Shows detailed information about a specific volume.

Syntax: `docker volume inspect volume_name`

Example: `docker volume inspect myvol`

4) docker volume rm: Delete one or more volumes

Syntax: `docker volume rm vol_name`

Example: `docker volume rm myvol`

5) docker volume prune: Delete all the unused volumes

Syntax: `docker volume prune`

Running a container with a named volume:

`docker run -d --name container1 -v myvol:/var/lib/mysql mysql:latest`

Running a container with anonymous volume:

`docker run -d -v /app nginx`

Docker Networks

> Docker networking is a system that allows containers to communicate with each other, with host machine, and with external networks in a controlled, secure and structured way.

Purpose of Docker networks:

- > Learning Docker networks is important because containers are rarely used individually. In real-world applications, containers work together i.e, frontend containers, a backend container and database containers.
- > It helps us with troubleshooting the issues. If something fails, it may not be a coding problem but it might be a network issues. If you want to fix the issues then you should have some knowledge about docker networks.

Docker Networking commands:

- 1) docker network ls: Lists all the Docker networks available on the system.

syntax: `docker network ls`

- 2) docker network inspect: Displays detailed information about a specific network (IP range, containers connected, driver details....)

syntax: `docker network inspect <network_name>`

- 3) docker network create: Creates a custom network.

syntax: `docker network create [FLAGS] <network_name>`

Common flags:

- > `--driver`: Specifies the network driver (Bridge, Host, None)
- > `--subnet`: Defines the custom subnet
- > `--gateways`: Assigns gateway IPs

- 4) docker network connect: Connects an existing container to a network

syntax: `docker network connect <network> <container_name>`

- 5) docker network disconnect: Remove the container from a network.

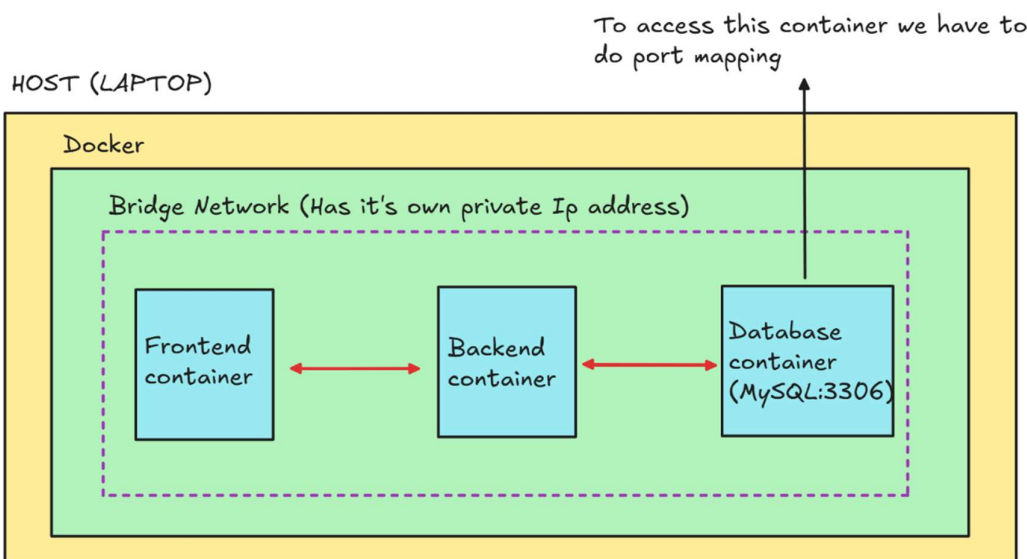
syntax: `docker network disconnect <network> <container_name>`

- 6) docker network rm: Delete a network

syntax: `docker network rm <network_name>`

1. Bridge networks:

- > Bridge network is default network provided by Docker.
- > A bridge network is a private internal network created on the host machine where containers can communicate with each other.
- > It isolates the containers from the external world unless ports are explicitly mapped.

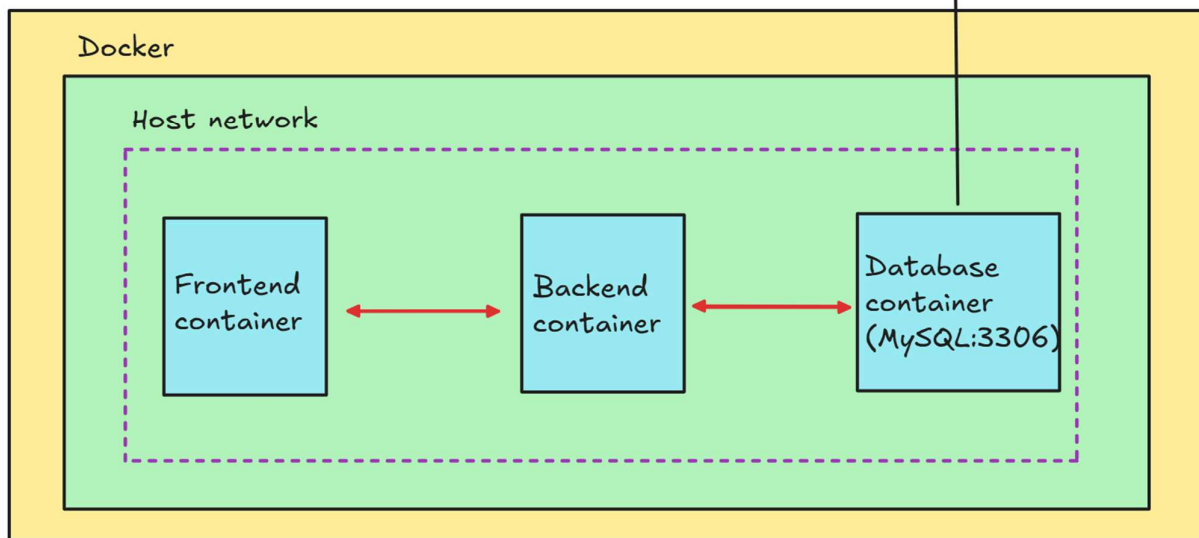


2. Host network:

- > A host network in Docker removes the virtual network isolation between the container and the host machine.
- > Instead of getting its own private IP address, the container uses, host machine's network.
- > It does not need port mapping.
- > Services running inside the container are directly accessible using the host's IP address or localhost.

HOST (LAPTOP)

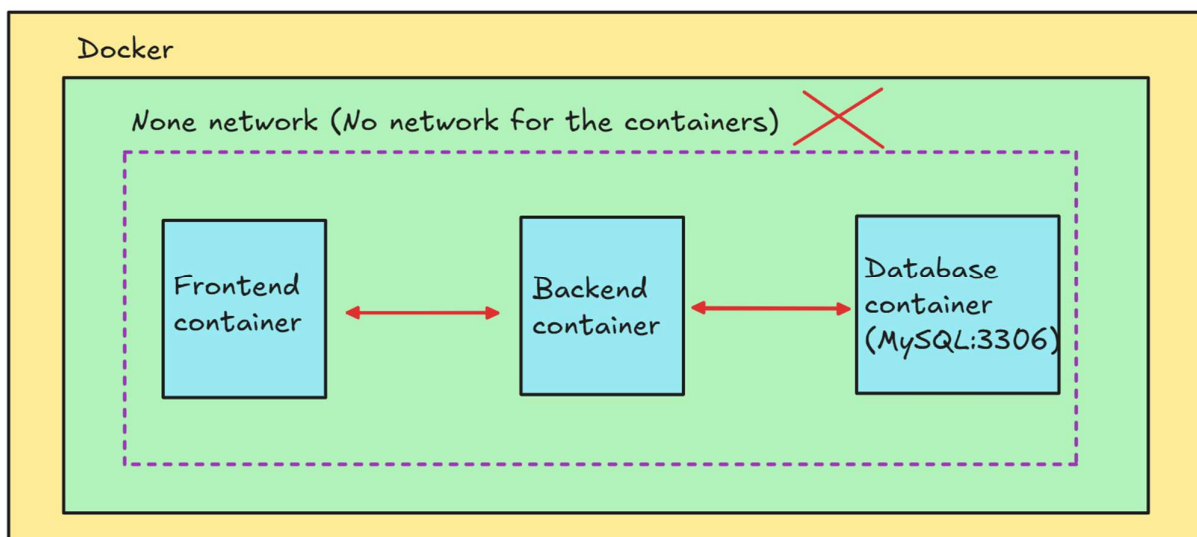
No need to do port mapping. Uses Host's network.



3. None network:

- > The none network in Docker is a special network mode where a container is created with no network access at all.
- > When a container uses the none network, it:
 - > Gets no IP address
 - > Cannot communicate with other containers
 - > Does not map to any ports.

HOST (LAPTOP)



Docker Compose

> Docker compose is a tool used to define and run multiple containers as a single application.

Docker compose commands:

1. docker compose up: This command is used to create and start all services defined in the yaml file.

syntax: `docker compose up`

2. docker compose down: This command is used to stops and removes all services defined in the yaml file.

syntax: `docker compose down`

3. docker compose ps: This command is used to check the status all services defined in the yaml file.

syntax: `docker compose ps`

4. docker compose start: This command is used to start all services defined in the yaml file.

syntax: `docker compose start`

5. docker compose stop: This command is used to stop all services defined in the yaml file.

syntax: `docker compose stop`

6. docker compose logs: It is used to display the logs.

syntax: `docker compose logs [SERVICE]`

what is YAML?

> YAML stands for "yet another markup language"

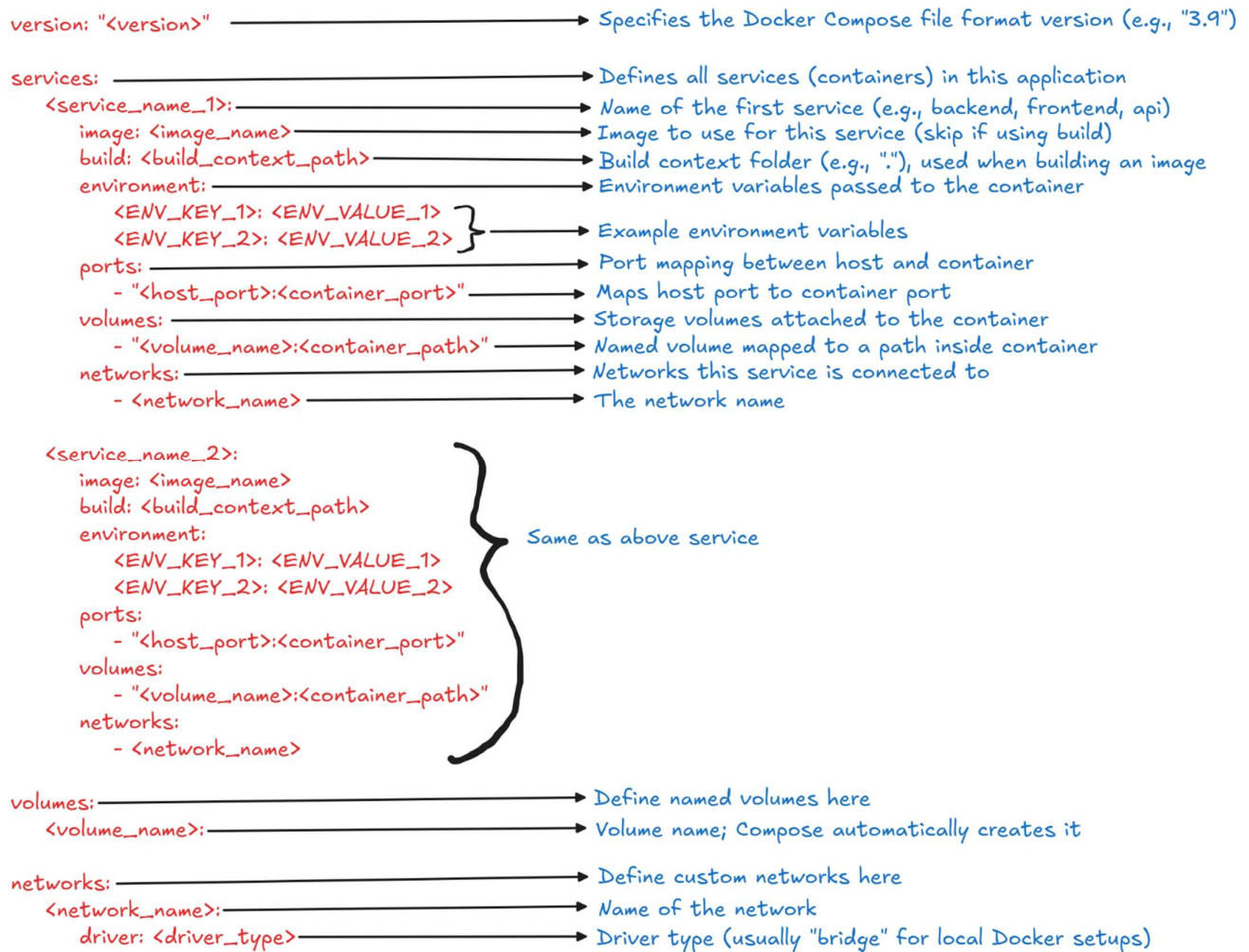
> It is a human readable data format.

> YAML focuses on readability and it uses indentation (spaces)

Example:

```
Country:
  India:
    State:
      Karnataka:
        - Bengaluru
        - Mangalore
        - Mysuru
      Maharashtra:
        - Mumbai
        - pune
```

Basic Syntax:



Example for docker compose for static web hosting

```
version: "1.0"
services:
  web:
    build: .
    ports:
      - "130:80"
```

Docker Integration with Jenkins

- 1) Clone spring-petclinic project and then write Dockerfile like this:

```
FROM eclipse-temurin:21.0.9_10-jdk
WORKDIR /demo
COPY target/*.jar /demo/petclinic.jar
EXPOSE 8080
ENTRYPOINT [ "java","-jar" ]
CMD ["petclinic.jar"]
```

- 2) After writing Dockerfile, create a .env file containing environment variables separately so that:

- > your Compose file stays clean
- > sensitive values are not written directly in docker-compose.yml
- > you can change environment values easily without modifying the compose file
- > you can reuse the same compose file in dev, test, and prod just by changing .env
- > You should not commit passwords into Git. Instead, .env stays in .gitignore.

MySQL settings

MYSQL_ROOT_PASSWORD=root123

MYSQL_USER=petclinic

MYSQL_PASSWORD=petclinic

MYSQL_DATABASE=petclinic

Spring Boot database config

SPRING_MYSQL_URL=jdbc:mysql://mysql:3306/petclinic

SPRING_MYSQL_USERNAME=petclinic

SPRING_MYSQL_PASSWORD=petclinic

Exposed port

SERVER_PORT=130

- 3) After creating .env file, write docker-compose.yaml file (You can either delete the already existing file or modify the same) like this:

version: "3.9"

services:

backend:

image: petclinic:latest

build: .

environment:

SPRING_MYSQL_URL: \${SPRING_MYSQL_URL}

SPRING_MYSQL_USERNAME: \${SPRING_MYSQL_USERNAME}

SPRING_MYSQL_PASSWORD: \${SPRING_MYSQL_PASSWORD}

ports:

- "\${SERVER_PORT}":8080

mysql:

image: mysql:8.0

environment:

MYSQL_ROOT_PASSWORD: \${MYSQL_ROOT_PASSWORD}

MYSQL_DATABASE: \${MYSQL_DATABASE}

MYSQL_USER: \${MYSQL_USER}

MYSQL_PASSWORD: \${MYSQL_PASSWORD}

ports:

- "3306:3306"

volumes:

- mysql-data:/var/lib/mysql

volumes:

mysql-data:

- 4) After this create a new repository and push the changes.
- 5) Log in to Jenkins and go to manage jenkins to install these plugins:
- > CloudBees Docker Build and Publish plugin
 - > Docker Pipeline
 - > Docker plugin
 - > CloudBees Docker Hub/Registry Notification
- 6) After installing the plugins, Open a new tab on a browser and search for Docker hub login. Login to your account and you will see the homepage. In the top left corner, you have an option called as "MyHub" (Next to Explore), click on it. Then click on create repository. Give the repository name, description and choose public or private based on your need and then click on create. Next you need to generate a token, for that click on top right corner for your Alphabet logo, and then select "Account settings". It will open a new tab, here go to the sidebar and there you will find an option called as "Personal access tokens". Click on "Generate new token".
- 8) Give the Access token description, Give any expiration date and for Access permission, select Read, Write, Delete finally click on generate. You will get a token, copy that and paste it somewhere in textfile.
- 9) Next go to Jenkins and then go to manage jenkins under that go to "Security" and then "Credentials" and then under "Stores scoped to Jenkins", click on "system". Then click on "Global credentials" and then click on "Add credentials".
- 10) Here you need to give your Docker credentials. Under like, select Secret Text, under Secret paste the token, under ID, write "docker-pass". Write the same for description and then click on Create.
- 11) Create a pipeline project and write this:

```

pipeline {
  agent any

  environment {
    DOCKERHUB_USER = "sumukha200147"
    IMAGE_NAME = "petclinic"
  }

  stages {
    stage('Git-Clone') {
      steps {
        git branch: 'main', url: 'https://github.com/Sumukha47/Docker-Spring-Petclinic.git'
      }
    }

    stage('Build') {
      steps {
        bat 'mvn clean package -DskipTests'
      }
    }

    stage('Build-Image') {
      steps {
        bat 'docker build -t %IMAGE_NAME%:latest .'
      }
    }

    stage('Pushing image to Dockerhub') {
      steps {
        withCredentials([string(credentialsId: 'dockerhub-pass', variable: 'TOKEN')]) {
          bat """
            echo %TOKEN% | docker login -u ${DOCKERHUB_USER} --password-stdin
            docker tag ${IMAGE_NAME} ${DOCKERHUB_USER}/${IMAGE_NAME}:latest
            docker push ${DOCKERHUB_USER}/${IMAGE_NAME}:latest
          """
        }
      }
    }

    stage('Create a container using Docker Compose') {
      steps {
        bat 'docker compose down || exit 0'
        bat 'docker compose up -d --build'
      }
    }
  }
}

```

To get this, Click on pipeline syntax:

- > Sample Step ---> withCredentials: Bind Credentials to variables
- > For "Bindings" click on +Add and select secret text.
- > Variable: TOKEN
- > Credentials: Choose the credentials
- > Click on Generate pipeline script

12) After writing this, click on Build now. If you go to Docker desktop, you can see that the image is built and container is running

ECR (Elastic Container Registry)

- > ECR stands for Amazon Elastic Container Registry. It is a fully managed container image registry provided by AWS.
- > It stores Docker images and lets you pull them when running containers on AWS services like ECS, EKS, Lambda, and EC2.

Types of Repositories in ECR

1. Private Repositories:

- > Private repositories are accessible only to your AWS account and to specific IAM users, roles, or services that you allow. They are secure, encrypted, and controlled by IAM permissions.
- > Access controlled by IAM
- > Cannot be accessed publicly
- > Best for production workloads
- > Encrypted by default
- > Supports image scanning for vulnerabilities

Use case examples:

- > Storing your company's application images
- > ECS, EKS, EC2 pulling images securely
- > CI/CD pipelines pushing images
- > Keeping internal or sensitive images

2. Public Repositories:

- > These repositories allow anyone on the internet to pull images without authentication.
- > They work similar to DockerHub public images, but with AWS-level reliability.
- > Publicly accessible without login
- > No rate limits like DockerHub
- > Backed by AWS global infrastructure
- > Good alternative to DockerHub public repos

Use case examples:

- > Sharing open-source images
- > Providing official company images
- > Hosting base images or public tools

Why do we need ECR if we already have DockerHub?

DockerHub is the most popular public container registry. It works for everyone and is great for open-source images. But when you deploy applications inside AWS, you need features that DockerHub cannot provide. Here are the real differences explained clearly:

1. Authentication:

- > DockerHub uses basic username and password or tokens.
- > ECR uses IAM permissions, AWS CLI, AWS roles, and temporary tokens.
- > Meaning: EC2, ECS, EKS machines can automatically access ECR without storing passwords.
- > Companies prefer private, secured registries. ECR provides IAM-based access control.
- > No plain passwords, no need to store DockerHub credentials on servers.

2. Security:

- > DockerHub is public by default. Private repos cost money and you only get one private repo for free tier account.
- > ECR provides private repositories by default, encrypted at rest, with full IAM security.

3. Speed and Performance:

- > DockerHub is global but not inside AWS.
- > ECR is inside AWS data centers.
- > When your application in AWS pulls the image from ECR, it is extremely fast, low latency, and uses AWS backbone network.
- > ECR integrates with AWS KMS (encryption) and CloudTrail (auditing).
- > This is important for enterprises following security standards.
- > Images download much faster inside AWS.
- > Your ECS cluster pulls an ECR image almost instantly.
- > CI/CD pipelines in AWS (CodeBuild, CodePipeline, GitHub Actions with AWS roles) work smoothly with ECR.

4. Reliability and Pull Limits:

- > DockerHub has download rate limits for free accounts.
- > ECR has no such restrictions for your AWS account.

Pull rate limits:

Docker Hub places limits on image download rates for free users, and exceeding these limits will cause requests to be blocked

Unauthenticated users: 100 pulls per 6 hours per IPv4 or IPv6 /64 subnet.

- > Authenticated free users: 200 pulls per 6 hours per IP address.
- > Paid users: Unlimited pull rate limits on Docker Hub per 6 hours.

Steps to push images to Amazon ECR:

- 1) Install AWS CLI to interact with AWS from your machine. Go to your browser and search for AWS CLI install for windows. scroll down and select windows and select "Download and run the AWS CLI MSI installer for Windows (64-bit)": <https://awscli.amazonaws.com/AWSCLIV2.msi>."
- 2) Go to the downloaded location and install it.
- 3) Login to AWS and search for IAM. Create one IAM user (Docker User) and give these permissions while creating the user:
 - > AmazonEC2ContainerRegistryFullAccess
 - > AmazonElasticContainerRegistryPublicFullAccess
- 4) After creating this user, click on the user you have created and under these options "Permissions", "Groups", "Tags(1)", "Security credentials", "Last Accessed" select Security credentials.
- 5) Scroll down till you get an option called as "Access keys" and click on Create access key.
- 6) Select "Command Line Interface (CLI): You plan to use this access key to enable the AWS CLI to access your AWS account." option, scroll down and tick on this "Confirmation: I understand the above recommendation and want to proceed to create an access key." and finally click on next and click on create access key. Click on "Download .csv file" and click on done.
- 7) Open VS code from your project folder and write these commands:

`aws configure`

when you give this command, it will ask you to fill these:

AWS Access Key ID [*****SSXV]:

AWS Secret Access Key [*****h+Gi]:

Default region name [us-east-1]: us-east-1

Default output format [json]: json

} Fill these credentials from the csv file which is downloaded.

→ Mention the region where you are creating your private repository (for public it is by default N.Virginia region)

→ Write it as Jason

- 8) After this, search for ECR and then click on create. By default it will take you to create a private repository. (If you want to create a public repository then in the sidebar select repositories under public registry.)
- 9) Under "General settings" give a repository name.
- 10) Under "Image tag settings" you have 2 options "Mutable" and "Immutable"

1. Mutable Tags: You can overwrite an existing image tag with a new image.

Example:

- > You have an image tagged as latest → version1
- > Later, you push a new image with the same tag: latest → now points to version2
- > The old image is replaced by the new one.

Why it is risky in production:

- > If ECS, EKS, or developers pull the same tag at different times, they may get different images. This causes unpredictable behavior.

2. Immutable Tags: You cannot overwrite, modify, or replace an existing tag once it is pushed.

Example:

- > If you push an image with tag: v1
- > You cannot push another image with the same tag v1.
- > You must create a new tag such as v2 or v1.1.

Why it is safer:

- > Guarantees consistency
- > Ensures same image is pulled every time
- > Avoids accidental overwrites
- > Better for production environments

- 11) Scroll down till the end and click on create.
- 12) Click on the Repository you have created and click on "View push commands" you will get list of commands with explanation. Copy these commands one by one and execute it in VS code inside the project folder (Note that Dockerfile should be present in that location).

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 462219490304.dkr.ecr.us-east-1.amazonaws.com
```

```
docker build -t sumukha-private-repository .
```

```
docker tag sumukha-private-repository:latest 462219490304.dkr.ecr.us-east-1.amazonaws.com/sumukha-private-repository:latest
```

```
docker push 462219490304.dkr.ecr.us-east-1.amazonaws.com/sumukha-private-repository:latest
```

- 13) After executing these commands, you can see that the image and it's related metadata is stored in ECR.